Лабораторна робота №1

Функціональне програмування

Завдання 3. Програмне забезпечення на мережі факультету

Левківський Євгеній Сергійович

ІПС-41

1. Мета роботи

Розробити інформаційну систему на Haskell з використанням класів типів та їхніх екземплярів. Спроєктувати базу MySQL, реалізувати введення, коригування, пошук і статистику використання програмного забезпечення факультету.

2. Постановка завдання

Обрано третє завдання: «Програмне забезпечення на мережі факультету». Таблиці повинні містити назву програмного засобу, авторів, анотацію, вид, версію, терміни та умови використання, місце зберігання дистрибутиву, відомості про користувачів, популярність і статистику використання. Кількість таблиць має бути більшою за п’ять.

Створено консольний застосунок. Код моделі та операцій міститься в lab1/src/Department.hs, інтерфейс команд — у lab1/app/Main.hs, SQL — у lab1/sql. Загальні назви Resource та resources позначають програмний ресурс; обов’язкове поле version зберігає його версію.

3. Структура MySQL

База faculty_software використовує utf8mb4 і містить 11 таблиць. Представлення resource_statistics не входить до кількості таблиць.

Таблиця	Дані
authors	Автори програмного забезпечення
resource_types	Види: компілятор, СУБД, навчальна програма
purposes	Призначення програм
usage_terms	Назва та текст умов використання
resources	Назва, версія, анотація, вид, умови, початок і кінець використання
resource_authors	Зв’язок програм та авторів багато-до-багатьох
resource_purposes	Зв’язок програм та призначень багато-до-багатьох
resource_urls	Адреси або локальні шляхи дистрибутивів
users	Користувач, група, унікальний логін
access_grants	Доступ користувача до програми
usage_events	Події використання, час і тривалість

Зовнішні ключі не дозволяють посилатися на неіснуючі довідники. Подія використання посилається на складений ключ доступу: запис про запуск без наданого доступу відхиляється БД. CHECK забороняє дату завершення раніше відкриття; NULL означає безстрокове використання. Залежні записи програми видаляються каскадно.

Популярність визначається кількістю подій використання. Представлення обчислює число запусків, кількість різних користувачів і сумарну тривалість. LEFT JOIN включає програми без запусків. Команда stats сортує результати за кількістю подій.

4. Класи та instances

Resource моделює отриманий запис, ResourceDraft — введені користувачем дані, MySQL — конфігурацію сервера й клієнта. Поля версії присутні в обох моделях і в операціях створення та оновлення.

```
class SqlValue a where
  sqlValue :: a -> String

class Validatable a where
  validate :: a -> Either String ()

class ResourceStore store where
  listResources :: store -> IO [Resource]
  searchResources :: store -> Text -> IO [Resource]
  insertResource :: store -> ResourceDraft -> IO Int
  updateResource :: store -> Int -> ResourceDraft -> IO Bool
  deleteResource :: store -> Int -> IO Bool
```

SqlValue має екземпляри для Text, Int, Day та Maybe a. Validatable ResourceDraft перевіряє непорожню назву й версію, анотацію, ідентифікатори та коректні календарні дати. ResourceStore MySQL реалізує CRUD і пошук. Це скорочений запис інтерфейсів; повний код та імпорти включені до комплекту.

Пошук охоплює назву, анотацію та авторів. Текст передається як UTF-8 hex-літерал CONVERT(X'...' USING utf8mb4), тому лапки чи SQL-оператори у введенні залишаються даними. Для команди edit таблиці та стовпці перевіряються за дозволеним списком.

5. Взаємодія з сервером і запуск

Система використовує справжній MySQL Server 8.4.6. Haskell запускає офіційний клієнт mysql як транспорт, передає SQL через stdin і читає результат зі stdout. Моделі, валідація, побудова запитів, інтерфейс та обробка помилок реалізовані Haskell.

Параметри MYSQL_BIN, MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_DATABASE задають клієнт і з’єднання. Пароль mysql читає з MYSQL_PWD. Локальний сценарій сам запускає підготовлений сервер та завантажує схему з прикладами.

```
node scripts/local.mjs lab1 list
node scripts/local.mjs lab1 search GHC
node scripts/local.mjs lab1 details 1
node scripts/local.mjs lab1 stats
```

Команди add та update приймають назву, версію, анотацію, вид, умови й період використання. Додаткові команди створюють довідники та користувачів, зв’язують авторів і призначення, додають дистрибутиви,

надають доступ і реєструють використання. Edit коригує довідники, користувачів та адреси. Приклади команд наведено в lab1/README.md.

6. Результати перевірки

Демонстраційна база містить GHC 9.6.7, MySQL Community Server 8.4.6 та навчальний аналізатор тексту 1.0.0. Для GHC зафіксовано 2 події, 1 користувача та 480 секунд. Для MySQL — 1 подія та 240 секунд, для аналізатора — 0 подій.

Перевірка	**Результат**
Список, пошук, довідники й дистрибутиви	Успішно
Створення програми версії 1.0	Збережено
Оновлення до версії 2.0	Знайдено оновлений запис
Автор, призначення, доступ і запуск 90 секунд	Статистика: 1 подія, 1 користувач, 90 секунд
Лапки, табуляція й новий рядок	Збережено як текст
SQL-ін’єкція та невідома таблиця edit	Небезпечний запит не виконується
Некоректна дата чи відсутній зовнішній ключ	Операцію відхилено
Видалення тестової програми	Залежні записи видалено

Перевірено на Windows x64, GHC 9.6.7 та справжньому сервері MySQL. Повний журнал: verification/lab1.txt. Сукупний набір автоматичних тестів трьох робіт містить 44 перевірки; усі пройшли.

7. Висновок

Виконано третє завдання: створено облік програмного забезпечення факультету з версіями, дистрибутивами, користувачами й популярністю. Реалізовано 11 таблиць MySQL, введення й коригування даних, пошук, статистику, класи SqlValue, Validatable, ResourceStore та відповідні instances. Роботу підтверджено автоматизованими перевірками.

8. Джерела

GHC User’s Guide: https://downloads.haskell.org/ghc/9.6.7/docs/users_guide/

MySQL 8.4 Reference Manual: https://dev.mysql.com/doc/refman/8.4/en/